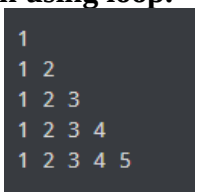
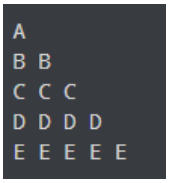
Pacific School of Engineering

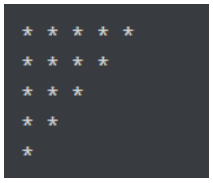
Department of Computer Engineering
Subject: PPS (3110003)

Most Important Questions of PPS with Solutions

1. **Explain algorithm and flowchart in detail.**
2. **Difference between Compiler vs Interpreter.**
3. **What is C? and Explain the structure of C program with example.**
4. **Explain features of C.**
5. **What is identifier? Rules for constructing identifiers?**
6. **List the data types provided by C programming language.**
7. **Explain Types of Operators in C with Example.**
8. **Explain – If..else, Nested If..else and else..if ladder Statement with example.**
9. **Explain Break and Continue Statement.**
10. **Explain Switch Statement**
11. **Explain looping Statements**
12. **What is Function in C? Explain Advantage of Function and Types of Function.**
13. **Explain Different aspects of function calling**
14. **what is string? Explain string handling function with example.**
15. **Explain structure in Detail.**
16. **Difference between Structure and Union.**
17. **Explain Dynamic memory allocation. Functions of Dynamic Memory Allocation.**
18. **Explain input output function with example.**
19. **Explain Pointer in C programming.**
20. **Print following Pattern using loop.**







21. **Write a program to evaluate the following series**
    1. **1^2+2^2+3^2+……+n^2**
    2. **1+1/2+1/3+1/4+....+1/n**

22. **Explain Pointer in C programming .**
23. **Explain Call by value and Call by reference in C.**

Q1 Explain algorithm and flowchart in detail.

Algorithm: The algorithm is a set of ordered instructions that are written in simple English Language.

- The algorithm defines the step by step logic for a program to solve specific problem.
- In this type of problem-solving the required inputs and expected outputs are identified and according to that the processing will be done.
- **Write an algorithm for accepting two numbers and performing addition.**

  Input: Two numbers

  Output : Addition of two numbers

  Step 1 Start

  Step 2 Declare variables NUMI, NUM2, ADD,

  Step 3 Read values for NUMI and NUM2 Add NUMI and NUM2 and assign the result to ADD. ADD-NUMI + NUM2

  Step 4 Stop

❖ **Flowchart :**

- The flowchart is a graphical representation of an algorithm.
- The sequence of steps in the algorithm is maintained and represented in the flowchart by using some standard symbols such as rectangles and directed lines.
- As flowchart uses the pictorial representation of an algorithm
- **Start/End terminal**

- Indicates the start or end of the algorithm. Represented as rounded rectangle.
- **Process**

- Indicates set of operation like Increment , Decrement.

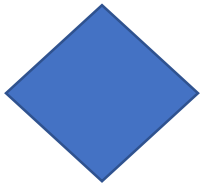
- Represent Symbol is Rectangle.
- **Input output**

- Indicates the inputs provided to the algorithm and outputs produced by algorithm.
- Represented the parallelogram.

- **Flowline**

- Indicates the flow of algorithm. Represented as an arrowline. It will connected two symbols in some order.

- **Decision**



- Decision make statement, condition which decision made symbol : Dimond
- **On page connector**

- Connecting part of flow on same page
- **Off page connector**

Connecting part of flow on different page

- **Predefine process**

Set of operation performed by one process which is defined elsewhere.

Function call

Q2 Difference between Compiler vs Interpreter.

Parameter	Compiler	Interpreter
Parameter	An entire program is taken by compiler at a time	A single line of code or instruction is taken by interpreter time
Input	It outputs intermediate object code.	It does not generate any intermediate object code.
Output	Faster	Comparatively slower
Speed	The compilation is implemented prior to execution.	Compilation and execution implemented simultaneously
Working	Shows all the errors after compilation	Displays line by line. errors
Errors	Comparatively difficult	Easier
Error detection	C, C++, C#, Scala	Java, PHP, Perl, Python, Ruby

Q3 What is C? and Explain the structure of C program with example.

- C is a general purpose structured programming language designed and written by Dennis Ritchie at AT & T's Bell Laboratories of USA in 1972.
- C is a middle level language because it combines the features of both high-level language as well as low- level language (like Assembly Language).
- C is very powerful programming language as well as it is more popular and widely used language. Since last 4 decades the popularity of C exists because it is reliable, simple and easy to use.

Structure of C program with example.

A C program basically consists of the following part:

1. **Documentation Section**

   Documentation Section contains the information about the program and its author which is provided in comments.

   It contains the name and other details of program and name of the programmer.

This section can be skipped by learners. But it is good practice to include this section by programmers while working on projects in team.

2. **Pre-processor**

   The pre-processor command tells a C compiler to include required library files. These files are included before compiling the code.

3. **Function Section**

   As mentioned earlier the C is block structured language. It uses functions to divide the code into blocks.

   Each C program should contain exactly one main() function from where the program execution begins. So we can say that each C program has at least one function.

4. **Declaration Section**

   If C program uses any data variables then those will be declared in this section.

5. **Statements and Expressions**

   Statements and expressions are usually written in the function block. Each statement ends with the semicolon (;)

6. **Comments**

   The comment is ignored by the compiler and it is used to add additional information about the program or its statements. It can be written anywhere in the program. It always good practice to write comments.

Structure of a C program

```
#include <stdio.h>

void main (void)
{
        printf("\nHello World\n");
}
```

Preprocessor directive (header file)

Program statement

```
#include <stdio.h>
#define VALUE 10

int global_var;

void main (void)
{
        /* This is the beginning of the program */

        int local_var;
        local_var = 5;
        global_var = local_var + VALUE;

        printf ("Total sum is: %d\n", global_var); // Print out the result
}
```

Preprocessor directive

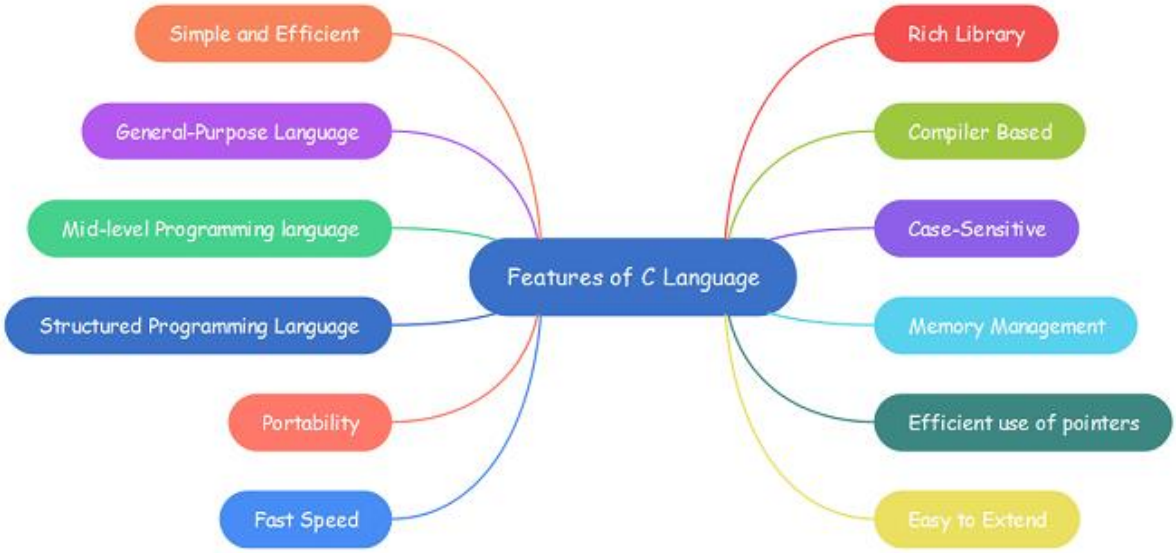
Global variable declaration

Comments

Local variable declaration

Variable definition

Q.4 Explain features of C.



1. Simple

- C uses English language to write the code, that's why it is very easy to write code in C.
- Also the syntaxes used in C coding are simple which can be easily understood by the programmer.

2. General purpose language

- Unlike COBOL and PASCAL, C is used for developing applications for all the purposes.
- C can be used for several domains of applications like, business and scientific applications.
- It has the ability to operate the bits, bytes and addresses means it can manipulate the memory hence it is used for system programming also.

3. Block structured language

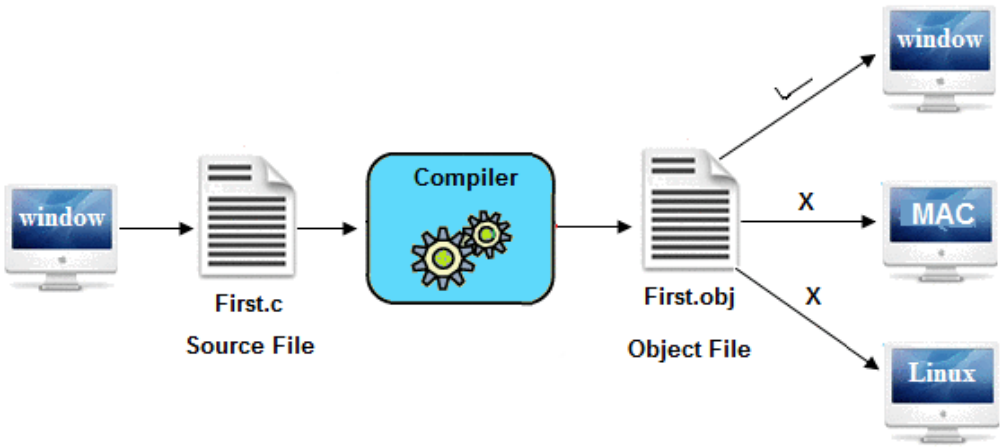
- As compare to other languages C is well structured language.
- The program code can be divided into several functions or blocks.
- The block structure approach helps to simplify the complexity of program by splitting it into smaller parts.
- To clearly understand the complex code, it is divided into blocks which can be manipulated by using control structure.

4. Middle level language

- C combines features of both high-level language and low-level language, so it is called as middle level language.
- C is near to machine as well as human as it deals with bit level operations and also uses English language while coding.

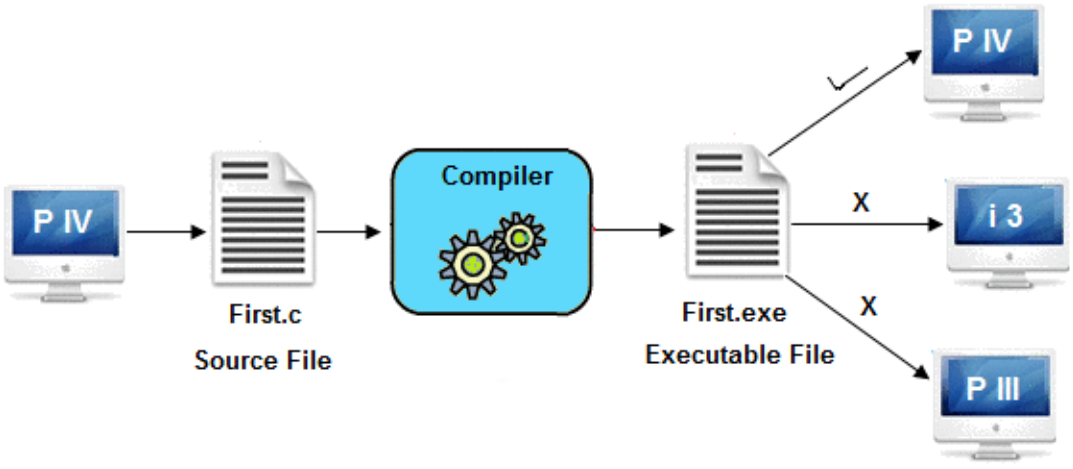
5. Platform dependent

- C is a platform dependent language.
- Platform dependent means the program runs on same operating system where it is developed and compiled, and cannot be executed on another operating system.
- The compiled version of a program file is platform dependent.

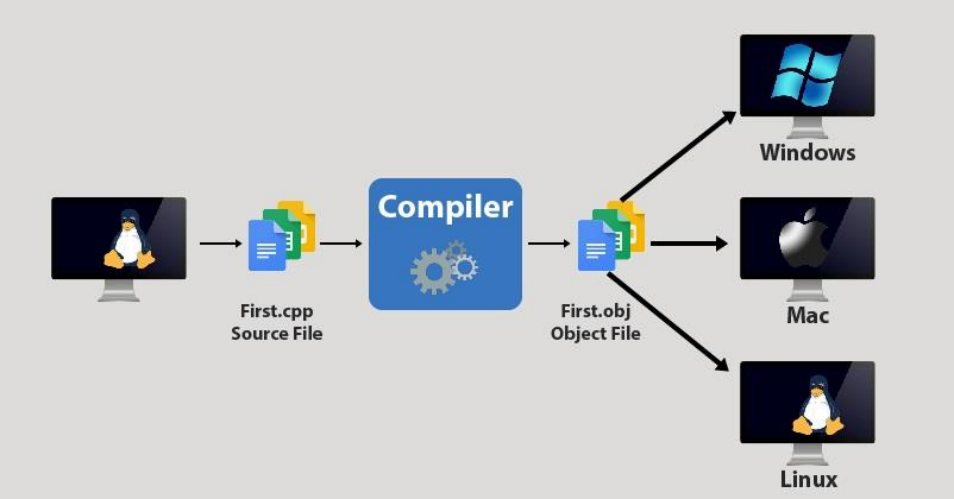


6. Portable

- Portability means executing the same application on different systems.
- In C Language three files are generated; the source code file (has .c extension), the compiled file (has .obj extension) and the executable file (has.exe extension).
- .exe file is not editable like .c file.
- The programs written and compiled on one operating system say Windows can be able to run on other windows based systems. This much portability is supported by C.



Note that this .exe file will not execute on other than windows operating system.



7. **Powerful**

- C has rich function library with built-in functions.
- Wide range of operators and data types are available in C. Also C provides various keywords.
- These all things make the C language more powerful.

8. **Case sensitive**

- C language is the case sensitive language so it consider int and INT as two different terms.

9. **Syntax based**

- C provides elegant syntax. While coding, the programmer should follow the syntax.

10. **Compiler based**

- Every program-written in C should be compiled first before execution. Without compilation the code will not execute because C is compiler based programming language.

11. **Fast and Efficient**

- Due to the large set of data types and operators, C is faster and efficient than other languages like BASIC, etc.

12. **Easily Extendable**

- C allows the programmer to use library functions and also add their own functions in library so as to extend and reuse the functions.

Q.5 What is identifier? Rules for constructing identifiers?

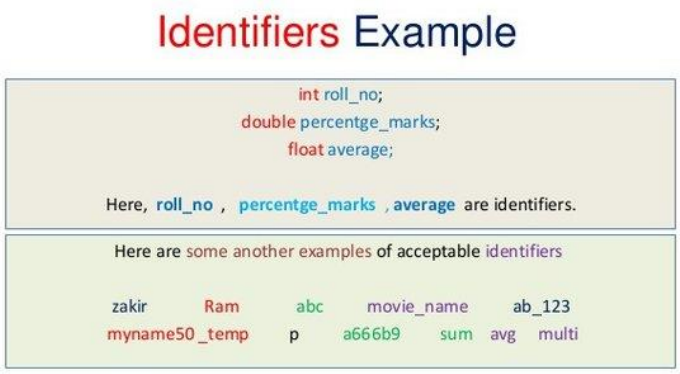
What is identifier?

- Identifier is a collection of alphanumeric characters. Identifiers are used to give name to the programming elements like variables, arrays, functions, structures, unions, and labels.
- Identifier is formed by combination of uppercase letters, lowercase letters, digits, and the underscore symbol.
- Basically, an identifier is user-defined word.
- It is valid to use any number and sequence of characters from 63 alphanumeric characters (among those 52 characters are uppercase and lowercase alphabets and 10 characters are 0 to 9 digits and 1 character is the underscore symbol) from the C character set to form an identifier.
- Identifiers start with the alphabets or underscore symbol.
- Usually, the underscore is used to separate two words in the long identifiers.

Rules for constructing identifiers

- Each identifier should start with an alphabet or underscore and then followed by any number of alphabets, digits or underscores.
- Identifier should not start with digit.
- Identifier must be a unique name given to the elements of a program.
- Identifiers are case sensitive so the max and MAX are considered as two different identifiers.
- An identifier should not contain special symbols, comma or blank space.
- Identifiers are user-defined words so they should not be same as of the keywords otherwise compiler will generate an error.
- There is no rule on the length of an identifier but first 31 characters are used by the compiler to recognize the uniqueness of it.

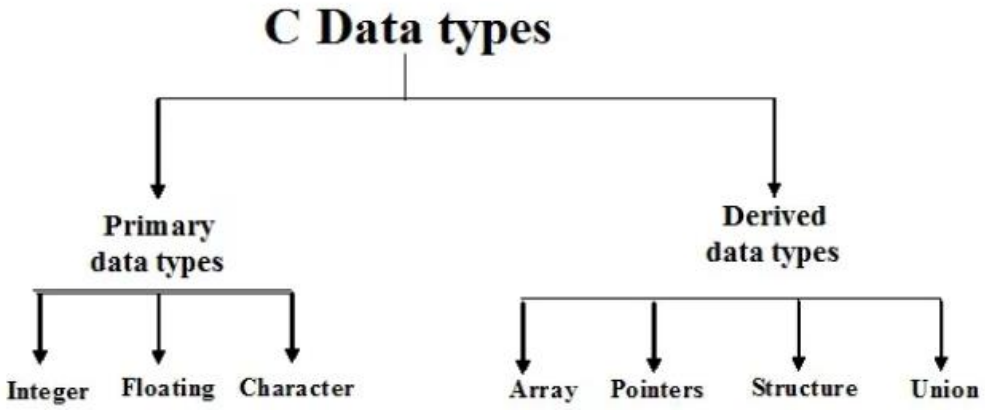
- Identifiers must be meaningful names given to the elements; it should be short and specific so that it will be easily understandable and remembered by the programmers.



Some other Example

Q.6 List the data types provided by C programming language

- A program may use different types of data for example, digit, character, real numbers, etc.
- Data types are used to specify the compiler which types of data the program manipulates.
- C language has some predefined set of data types to handle various kinds of data that we use in our program. These data types have different storage capacities.



C Several data types are broadly divided into two categories.

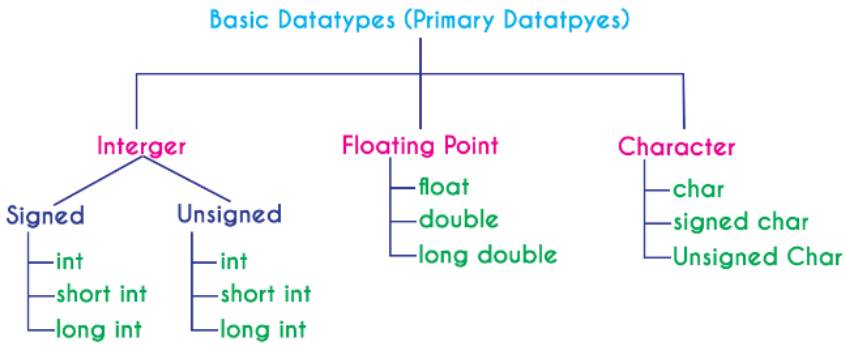
1. Primary data types

These are basic data types. int, char, float, void are the primary data types.

2. Derived data types

Derived data types are derived from primary data types. Array, functions, structure, union and pointers are the derived data types.

Primary data types



1. **Integer type**

Integers are used to store whole numbers like 1,0, 15, 3049,

The integer data type is a set of whole numbers. Every integer value does not have the decimal value. We use the keyword "**int**" to represent integer data type in c. We use the keyword int to declare the variables and to specify the return type of a function. The integer data type is used with different type modifiers like short, long, signed and unsigned. The following table provides complete details about the integer data type.

Type	Size (Bytes)	Range	Specifier
int (signed short int)	2	-32768 to +32767	%d
short int (signed short int)	2	-32768 to +32767	%d
long int (signed long int)	4	-2,147,483,648 to +2,147,483,647	%d
unsigned int (unsigned short int)	2	0 to 65535	%u
unsigned long int	4	0 to 4,294,967,295	%u

2. **Floating type**
   - Floating-point data types are a set of numbers with the decimal value. Every floating-point value must contain the decimal value. The floating-point data type has two variants...

float

double

- We use the keyword "**float**" to represent floating-point data type and "**double**" to represent double data type in c. Both float and double are similar but they differ in the number of decimal places. The float value contains 6 decimal places whereas double value contains 15 or 19 decimal places. The following table provides complete details about floating-point data types.

Floating types are used to store real numbers.

Type	Size (Bytes)	Range	Specifier
float	4	1.2E - 38 to 3.4E + 38	%f
double	8	2.3E-308 to 1.7E+308	%ld
long double	10	3.4E-4932 to 1.1E+4932	%ld

3. **Character type**

The character data type is a set of characters enclosed in single quotations. The following table provides complete details about the character data type.

Character types are used to store characters value.

Type	Size (Bytes)	Range	Specifier
char (signed char)	1	-128 to +127	%c
unsigned char	1	0 to 255	%c

4. **void type**

   void type means no value. This is usually used to specify the type of functions

	Integer	Floating Point	Double	Character
What is it?	Numbers without decimal value	Numbers with decimal value	Numbers with decimal value	Any symbol enclosed in single quotation
Keyword	int	float	double	char
Memory Size	2 or 4 Bytes	4 Bytes	8 or 10 Bytes	1 Byte
Range	-32768 to +32767 (or) 0 to 65535 (Incase of 2 bytes only)	1.2E - 38 to 3.4E + 38	2.3E-308 to 1.7E+308	-128 to + 127 (or) 0 to 255
Type Specifier	%d or %i or %u	%f	%ld	%c or %s
Type Modifier	short, long signed, unsigned	No modifiers	long	signed, unsigned
Type Qualifier	const, volatile	const, volatile	const, volatil	const, volatile

Q. 7 Explain Types of Operators in C with Example.

Operators are the foundation of any programming language. We can define operators as symbols that help us to perform specific mathematical and logical computations on operands. In other words, we can say that an operator operates the operands. For example, '+' is an operator used for addition, as shown below:

```
c = a + b;
```

Here, '+' is the operator known as the addition operator and 'a' and 'b' are operands. The addition operator tells the compiler to add both of the operands 'a' and 'b'.

- **C/C++ has many built-in operators and can be classified into 6 types:**

Arithmetic Operators

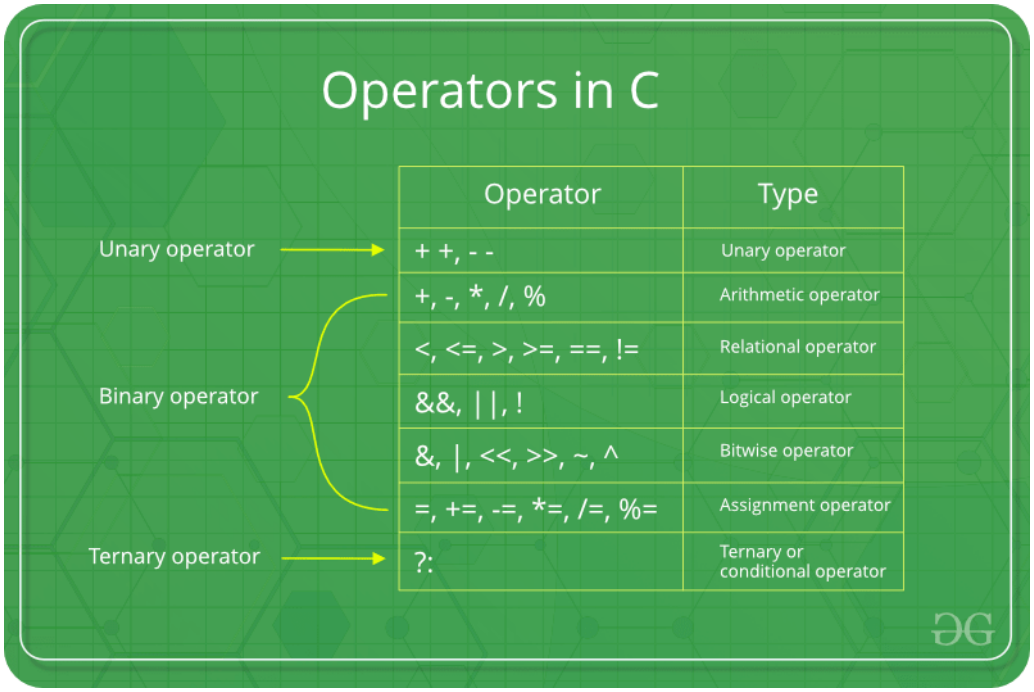
Relational Operators

Logical Operators

Bitwise Operators

Assignment Operators

Other Operators



1. C Arithmetic Operators

An arithmetic operator performs mathematical operations such as addition, subtraction, multiplication, division etc on numerical values (constants and variables).

Operator	Meaning of Operator
+	addition or unary plus
-	subtraction or unary minus
*	multiplication
/	division
%	remainder after division (modulo division)

Example 1: Arithmetic Operators

```c
// Working of arithmetic operators
#include <stdio.h>
int main()
{
    int a = 9,b = 4, c;

    c = a+b;
    printf("a+b = %d \n",c);
    c = a-b;
    printf("a-b = %d \n",c);
    c = a*b;
    printf("a*b = %d \n",c);
    c = a/b;
    printf("a/b = %d \n",c);
    c = a%b;
    printf("Remainder when a divided by b = %d \n",c);

    return 0;
}
```

Output

```
a+b = 13
a-b = 5
a*b = 36
a/b = 2
Remainder when a divided by b=1
```

2. C Increment and Decrement Operators

- C programming has two operators increment ++ and decrement -- to change the value of an operand (constant or variable) by 1.
- Increment ++ increases the value by 1 whereas decrement -- decreases the value by 1. These two operators are unary operators, meaning they only operate on a single operand

Example 2: Increment and Decrement Operators

```c
// Working of increment and decrement operators
#include <stdio.h>
int main()
{
    int a = 10, b = 100;
    float c = 10.5, d = 100.5;

    printf("++a = %d \n", ++a);
    printf("--b = %d \n", --b);
    printf("++c = %f \n", ++c);
    printf("--d = %f \n", --d);

    return 0;
}
```

Output

```
++a = 11
--b = 99
++c = 11.500000
--d = 99.500000
```

Here, the operators ++ and -- are used as prefixes. These two operators can also be used as postfixes like a++ and a--.

3. C Relational Operators

- A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.

Operator	Meaning of Operator	Example
==	Equal to	`5 == 3` is evaluated to 0
>	Greater than	`5 > 3` is evaluated to 1
<	Less than	`5 < 3` is evaluated to 0
!=	Not equal to	`5 != 3` is evaluated to 1
>=	Greater than or equal to	`5 >= 3` is evaluated to 1
<=	Less than or equal to	`5 <= 3` is evaluated to 0

Example 4: Relational Operators

```c
// Working of relational operators
#include <stdio.h>
int main()
{
    int a = 5, b = 5, c = 10;

    printf("%d == %d is %d \n", a, b, a == b);
    printf("%d == %d is %d \n", a, c, a == c);
    printf("%d > %d is %d \n", a, b, a > b);
    printf("%d > %d is %d \n", a, c, a > c);
    printf("%d < %d is %d \n", a, b, a < b);
    printf("%d < %d is %d \n", a, c, a < c);
    printf("%d != %d is %d \n", a, b, a != b);
    printf("%d != %d is %d \n", a, c, a != c);
    printf("%d >= %d is %d \n", a, b, a >= b);
    printf("%d >= %d is %d \n", a, c, a >= c);
    printf("%d <= %d is %d \n", a, b, a <= b);
    printf("%d <= %d is %d \n", a, c, a <= c);

    return 0;
}
```

Output

```
5 == 5 is 1
5 == 10 is 0
5 > 5 is 0
5 > 10 is 0
5 < 5 is 0
5 < 10 is 1
5 != 5 is 0
5 != 10 is 1
5 >= 5 is 1
5 >= 10 is 0
5 <= 5 is 1
5 <= 10 is 1
```

4. C Logical Operators

An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false. Logical operators are commonly used in decision making in C programming.

Operator	Meaning	Example
&&	Logical AND. True only if all operands are true	If c = 5 and d = 2 then, expression `((c==5) && (d>5))` equals to 0.
||	Logical OR. True only if either one operand is true	If c = 5 and d = 2 then, expression `((c==5) || (d>5))` equals to 1.
!	Logical NOT. True only if the operand is 0	If c = 5 then, expression `!(c==5)` equals to 0.

```c
#include <stdio.h>
int main()
{
    int a = 5, b = 5, c = 10, result;

    result = (a == b) && (c > b);
    printf("(a == b) && (c > b) is %d \n", result);

    result = (a == b) && (c < b);
    printf("(a == b) && (c < b) is %d \n", result);

    result = (a == b) || (c < b);
    printf("(a == b) || (c < b) is %d \n", result);

    result = (a != b) || (c < b);
    printf("(a != b) || (c < b) is %d \n", result);

    result = !(a != b);
    printf("!(a != b) is %d \n", result);

    result = !(a == b);
    printf("!(a == b) is %d \n", result);

    return 0;
}
```

Output

```
(a == b) && (c > b) is 1
(a == b) && (c < b) is 0
(a == b) || (c < b) is 1
(a != b) || (c < b) is 0
!(a != b) is 1
!(a == b) is 0
```

5. C Bitwise Operators

- During computation, mathematical operations like: addition, subtraction, multiplication, division, etc are converted to bit-level which makes processing faster and saves power.
- Bitwise operators are used in C programming to perform bit-level operations.

Operators	Meaning of operators
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise complement
<<	Shift left
>>	Shift right

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bit Operation of 12 and 25
  00001100
& 00011001
  _________
  00001000  = 8 (In decimal)
```

Example #1: Bitwise AND

```c
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a&b);
    return 0;
}
```

Output

```
Output = 8
```

Bitwise OR operator |

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bitwise OR Operation of 12 and 25
  00001100
| 00011001
  ________
  00011101  = 29 (In decimal)
```

Example #2: Bitwise OR

```c
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a|b);
    return 0;
}
```

Output

```
Output = 29
```

Bitwise XOR

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bitwise OR Operation of 12 and 25
  00001100
| 00011001
  ________
  00011101  = 29 (In decimal)
```

Example #2: Bitwise OR

```
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a|b);
    return 0;
}
```

Output

```
Output = 29
```

6. Comma Operator

Comma operators are used to link related expressions together. For example:

```
int a, c = 5, d;
```

Q.8 Explain – If..else, Nested If..else and else..if ladder Statement with example.

C If else statement

Syntax of if else statement:

- If condition returns true then the statements inside the body of "if" are executed and the statements inside body of "else" are skipped.
- If condition returns false then the statements inside the body of "if" are skipped and the statements in "else" are executed.

```
if(condition) {
   // Statements inside body of if
}
else {
   //Statements inside body of else
}
```

Flow diagram of if else statement

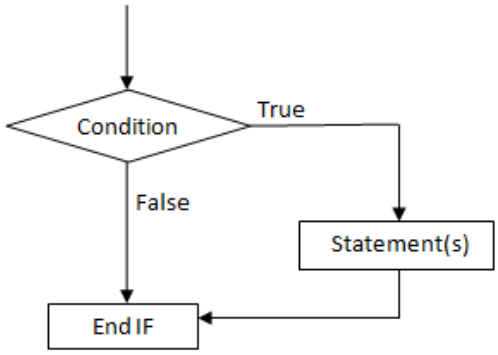


fig: Flowchart for if statement

Example of if else statement:

In this program user is asked to enter the age and based on the input, the if..else statement checks whether the entered age is greater than or equal to 18. If this condition meet then display message "You are eligible for voting", however if the condition doesn't meet then display a different message "You are not eligible for voting".

```c
#include <stdio.h>
int main()
{
   int age;
   printf("Enter your age:");
   scanf("%d",&age);
   if(age >=18)
   {
        /* This statement will only execute if the
         * above condition (age>=18) returns true
         */
        printf("You are eligible for voting");
   }
   else
   {
        /* This statement will only execute if the
         * condition specified in the "if" returns false.
         */
        printf("You are not eligible for voting");
   }
   return 0;
}
```

Output:

```
Enter your age:14
You are not eligible for voting
```

C Nested If..else statement

When an if else statement is present inside the body of another “if” or “else” then this is called nested if else.

Syntax of Nested if else statement:

```c
if(condition) {
    //Nested if else inside the body of "if"
    if(condition2) {
       //Statements inside the body of nested "if"
    }
    else {
       //Statements inside the body of nested "else"
    }
}
else {
    //Statements inside the body of "else"
}
```

Example of nested if..else

```c
#include <stdio.h>
int main()
{
   int var1, var2;
   printf("Input the value of var1:");
   scanf("%d", &var1);
   printf("Input the value of var2:");
   scanf("%d",&var2);
   if (var1 != var2)
   {
        printf("var1 is not equal to var2\n");
        //Nested if else
        if (var1 > var2)
        {
                printf("var1 is greater than var2\n");
        }
        else
        {
                printf("var2 is greater than var1\n");
        }
   }
   else
   {
        printf("var1 is equal to var2\n");
   }
   return 0;
```

Output:

```
Input the value of var1:12
Input the value of var2:21
var1 is not equal to var2
var2 is greater than var1
```

C – else..if statement

The else..if statement is useful when you need to check multiple conditions within the program, nesting of if-else blocks can be avoided using else..if statement.

Syntax of else..if statement:

```
if (condition1)
{
   //These statements would execute if the condition1 is true
}
else if(condition2)
{
   //These statements would execute if the condition2 is true
}
else if (condition3)
{
   //These statements would execute if the condition3 is true
}
.
.
else
{
   //These statements would execute if all the conditions return false.
}
```

Example of else..if statement

```
#include <stdio.h>
int main()
{
   int var1, var2;
   printf("Input the value of var1:");
   scanf("%d", &var1);
   printf("Input the value of var2:");
   scanf("%d",&var2);
   if (var1 !=var2)
   {
        printf("var1 is not equal to var2\n");
   }
   else if (var1 > var2)
   {
        printf("var1 is greater than var2\n");
   }
   else if (var2 > var1)
   {
        printf("var2 is greater than var1\n");
   }
   else
   {
        printf("var1 is equal to var2\n");
   }
   return 0;
}
```

Output:

```
Input the value of var1:12
Input the value of var2:21
var1 is not equal to var2
```

Q 9 Explain Break and Continue Statement.

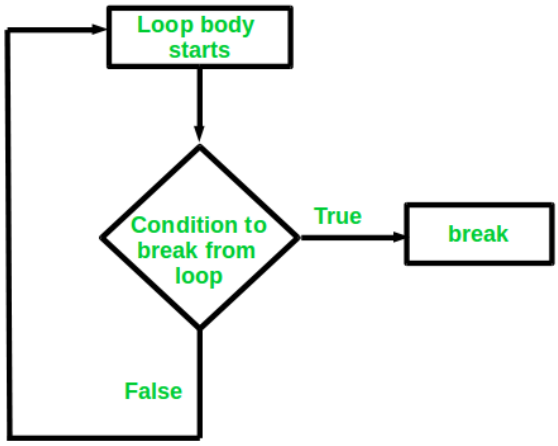
C break

The break is a keyword in C which is used to bring the program control out of the loop. The break statement is used inside loops or switch statement. The break statement breaks the loop one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops. The break statement in C can be used in the following two scenarios:

1. With switch case
2. With loop

Syntax:

```
//loop or switch case
break;
```



```c
#include<stdio.h>
#include<stdlib.h>
void main ()
{
   int i;
   for(i = 0; i<10; i++)
   {
      printf("%d ",i);
      if(i == 5)
      break;
   }
   printf("came outside of loop i = %d",i);

}
```

Output

```
0 1 2 3 4 5 came outside of loop i = 5
```

C continue statement

The continue statement in C language is used to bring the program control to the beginning of the loop. The continue statement skips some lines of code inside the loop and continues with the next iteration. It is mainly used for a condition so that we can skip some code for a particular condition.

Syntax:

```c
//loop statements
continue;
//some lines of the code which is to be skipped
```

Continue statement example

```c
#include<stdio.h>
int main(){
int i=1;//initializing a local variable
//starting a loop from 1 to 10
for(i=1;i<=10;i++){
if(i==5){//if value of i is equal to 5, it will continue the loop
continue;
}
printf("%d \n",i);
}//end of for loop
return 0;
}
```

Output

```
1
2
3
4
6
7
8
9
10
```

As you can see, 5 is not printed on the console because loop is continued at i==5.

Q 10 Explain Switch Statement

Instead of writing **many** if..else statements, you can use the switch statement.

The switch statement selects one of many code blocks to be executed:

Syntax

```c
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

This is how it works:

- The switch expression is evaluated once
- The value of the expression is compared with the values of each case
- If there is a match, the associated block of code is executed
- The break statement breaks out of the switch block and stops the execution
- The default statement is optional, and specifies some code to run if there is no case match

```
int a = 9;
switch (a) {
 case 1: printf("I am One\n");
         break;
 case 2: printf("I am Two\n");
         break;
 case 3: printf("I an Three\n");
         break;
 case 4: printf("I am Four\n");
         break;
 case 5: printf("I am Five\n");
         break;
 case 6: printf("I am Six\n");
         break;
 case 7: printf("I am Seven\n");
         break;
 case 8: printf("I am Eight\n");
 case 9: printf("I am Nine\n");
 case 0: printf("I am Zero\n");
 default: printf("I am default\n");
}
```

Q 11 Explain looping Statements

What is Loop in C?

Looping Statements in C execute the sequence of statements many times until the stated condition becomes false. A loop in C consists of two parts, a body of a loop and a control statement. The control statement is a combination of some conditions that direct the body of the loop to execute until the specified condition becomes false. The purpose of the C loop is to repeat the same code a number of times.

Types of Loops in C

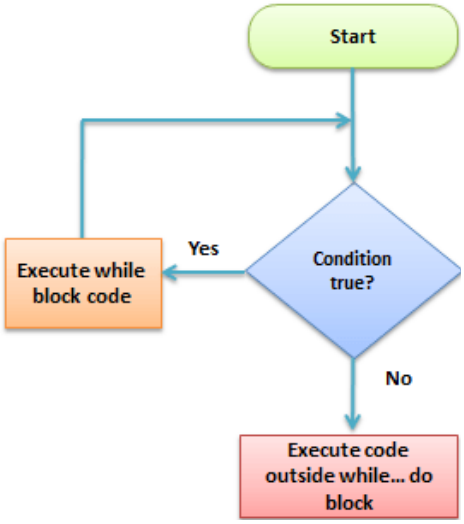
Depending upon the position of a control statement in a program, looping statement in C is classified into two types:

1. Entry controlled loop

2. Exit controlled loop

In an entry control loop in C, a condition is checked before executing the body of a loop. It is also called as a pre-checking loop.

In an exit controlled loop, a condition is checked after executing the body of a loop. It is also called as a post-checking loop.



The control conditions must be well defined and specified otherwise the loop will execute an infinite number of times. The loop that does not stop executing and processes the statements number of times is called as an infinite loop. An infinite loop is also called as an "Endless loop." Following are some characteristics of an infinite loop:

1. No termination condition is specified.

2. The specified conditions never meet.

The specified condition determines whether to execute the loop body or not.

'C' programming language provides us with three types of loop constructs:

1. The while loop

2. The do-while loop

3. The for loop

Sr. No.	Loop Type	Description
1.	While Loop	In while loop, a condition is evaluated before processing a body of the loop. If a condition is true then and only then the body of a loop is executed.
2.	Do-While Loop	In a do…while loop, the condition is always executed after the body of a loop. It is also called an exit-controlled loop.
3.	For Loop	In a for loop, the initial value is performed only once, then the condition tests and compares the counter to a fixed value after each iteration, stopping the for loop when false is returned.

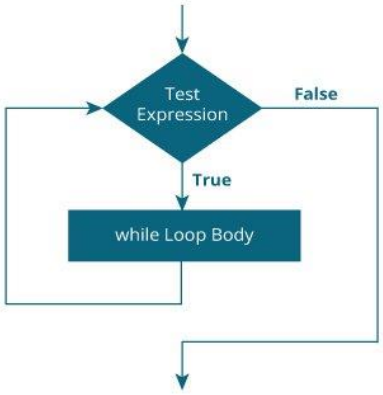
while loop

syntax

```
while (testExpression) {
  // the body of the loop
}
```

How while loop works?

- The while loop evaluates the testExpression inside the parentheses ().
- If testExpression is true, statements inside the body of while loop are executed. Then, testExpression is evaluated again.
- The process goes on until testExpression is evaluated to false.
- If testExpression is false, the loop terminates (ends).



Flowchart of while loop

Example 1: while loop

```
// Print numbers from 1 to 5

#include <stdio.h>
int main() {
  int i = 1;

  while (i <= 5) {
    printf("%d\n", i);
    ++i;
  }

  return 0;
}
```

Output

```
1
2
3
4
5
```

do...while loop

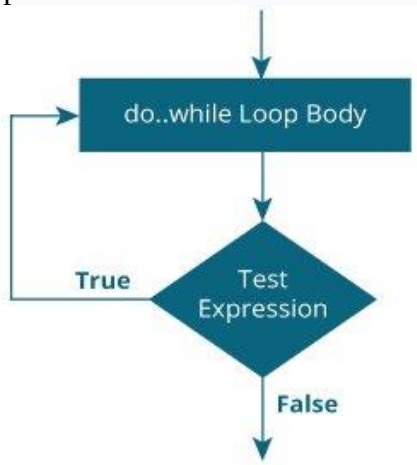
The do..while loop is similar to the while loop with one important difference. The body of do...while loop is executed at least once. Only then, the test expression is evaluated.

The syntax of the do...while loop is:

```
do {
  // the body of the loop
}
while (testExpression);
```

How do...while loop works?

- The body of do...while loop is executed once. Only then, the testExpression is evaluated.
- If testExpression is **true**, the body of the loop is executed again and testExpression is evaluated once more.
- This process goes on until testExpression becomes **false**.
- If testExpression is **false**, the loop ends.



Flowchart of do...while Loop

Example 2: do...while loop

```c
// Program to add numbers until the user enters zero

#include <stdio.h>
int main() {
  double number, sum = 0;

  // the body of the loop is executed at least once
  do {
    printf("Enter a number: ");
    scanf("%lf", &number);
    sum += number;
  }
  while(number != 0.0);

  printf("Sum = %.2lf",sum);

  return 0;
}
```

Output

```
Enter a number: 1.5
Enter a number: 2.4
Enter a number: -3.4
Enter a number: 4.2
Enter a number: 0
Sum = 4.70
```

for Loop

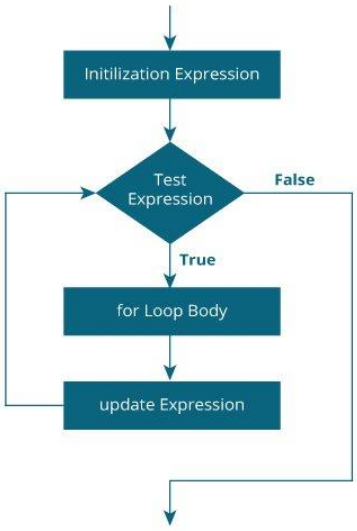
The syntax of the for loop is:

```c
for (initializationStatement; testExpression; updateStatement)
{
    // statements inside the body of loop
}
```

How for loop works?

- The initialization statement is executed only once.
- Then, the test expression is evaluated. If the test expression is evaluated to false, the for loop is terminated.
- However, if the test expression is evaluated to true, statements inside the body of the for loop are executed, and the update expression is updated.
- Again the test expression is evaluated.

This process goes on until the test expression is false. When the test expression is false, the loop terminates.



for loop Flowchart

Example 1: for loop

```c
// Print numbers from 1 to 10
#include <stdio.h>

int main() {
  int i;

  for (i = 1; i < 11; ++i)
  {
    printf("%d ", i);
  }
  return 0;
}
```

Output

```
1 2 3 4 5 6 7 8 9 10
```

Q 12 What is Function in C? Explain Advantage of Function and Types of Function.

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.
- **The function helps to simplify the complexity of program by splitting it into smaller parts called as functions .**
- **C provides library functions and also allows users to define their own functions .**

Need of Function

- Every C program uses main ( ) function . Sometimes the main ( ) function is not sufficient because program may become too long and complex as the number of instructions increases . It affects the result of debugging , testing and maintaining the program . So if the program is divided into functional parts , it will be easier to code , debug , test , and maintain the parts individually and then combine into single unit . These subparts are referred as functions .
- To repeat the set of statements we have used loops but in some situations set of operations needed to be performed at many different points throughout the program . For example , the operation of calculating the gross salary of employee needed repeatedly but at different times in the program . One approach is repeating the statements which calculate gross salary whenever required in the program . Another approachs to construct a function for calculating gross salary at once and call it whenever required in the program . Obviously second approach saves time and space so it is better .
- A situation where multiple calculations are needed in the program and the result of one calculation may be used as an input for another calculation . In this case program becomes very large and complicated .

Advantage of functions in C

- By using functions, we can avoid rewriting same logic/code again and again in a program.
- We can call C functions any number of times in a program and from any place in a program.

- We can track a large C program easily when it is divided into multiple functions.
- Reusability is the main achievement of C functions.
- However, Function calling is always a overhead in a C program.

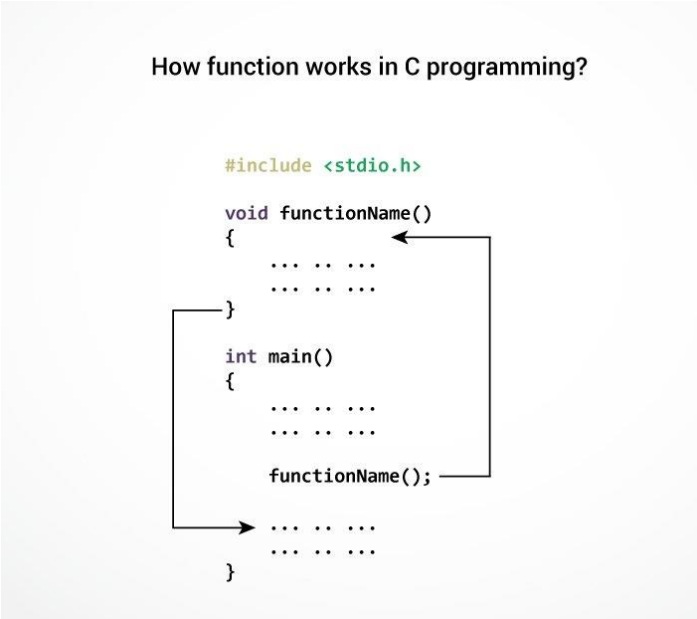
- **There are two types of function in C programming:**
  Standard library functions
  User-defined functions

Standard library functions

- The standard library functions are built-in functions in C programming.
- These functions are defined in header files. For example,
- The printf() is a standard library function to send formatted output to the screen (display output on the screen). This function is defined in the stdio.h header file. Hence, to use the printf()function, we need to include the stdio.h header file using #include <stdio.h>.
- The sqrt() function calculates the square root of a number. The function is defined in the math.h header file.

User-defined function

- You can also create functions as per your need. Such functions created by the user are known as user-defined functions.
- Programmer has to write the coding for such functions and test them properly before using them.
- The syntax of the function is given by the user so there is no need to include any header files.
- For example, main(), swap(), sum(), etc., are some of the user defined functions.



Q 13 Explain Different aspects of function calling

SN	C function aspects	Syntax
1	Function declaration	return_type function_name (argument list);
2	Function call	function_name (argument_list)
3	Function definition	return_type function_name (argument list) {function body;}

A function may or may not accept any argument. It may or may not return any value. Based on these facts, There are four different aspects of function calls.

1. function without arguments and without return value
2. function without arguments and with return value
3. function with arguments and without return value
4. function with arguments and with return value
5.

function without arguments and without return value

- When a function has no arguments, it does not receive any data from the calling function. Similarly when it does not return a value, the calling function does not receive any data from the called function.

```c
//No Return Without Argument Function in C
/*
    1.Function Declaration
    2.Function Definition
    3.Function Calling

*/
#include<stdio.h>

//Function Declaration
void add();

int main()
{
    //Function Calling
    add();
    return 0;
}
//Function Definition
void add()
{
    int a,b,c;
    printf("\nEnter The Value of A & B :");
    scanf("%d%d",&a,&b);
    c=a+b;
    printf("\nTotal : %d",c);
}
```

Output

```
Enter The Value of A & B :12
34

Total : 46
```

function without arguments and with return value

- There could be occasions where we may need to design functions that may not take any arguments but returns a value to the calling function. A example for this is getchar function it has no parameters but it returns an integer an integer type data that represents a character.

```c
//Return Without Argument Function in C
#include<stdio.h>

int add();

int main()
{
    int a;
    a=add();
    printf("\nTotal : %d",a);
    return 0;
}

int add()
{
    int a,b;
    printf("\nEnter The Value of A & B : ");
    scanf("%d%d",&a,&b);
    return a+b;
}
```

Output

```
Enter The Value of A & B : 6
45

Total : 51
```

function with arguments and without return value

- When a function has arguments, it receive any data from the calling function but it returns no values.

```c
//No Return With Argument Function in C
#include<stdio.h>

//Function Declaration
void add(int,int);

int main()
{
    int a,b;
    printf("\nEnter The Value of A & B : ")1
    scanf("%d%d",&a,&b);
    //Function Calling
    add(a,b);  // Actual Parameters
    return 0;
}
//Function Definition
void add(int x,int y)  //Formal Parameters
{
    int c;
    c=x+y;
    printf("\nTotal : %d",c);
}
```

Output

```
Enter The Value of A & B : 23
34

Total : 57
```

function with arguments and with return value

- Here this function is taking any input argument, and also returns value.

Source Code

```c
//Return With Argument Function in C
#include<stdio.h>
int add(int,int);
int main()
{
    int a,b;
    printf("\nEnter The Value of A & B : ");
    scanf("%d%d",&a,&b);
    a=add(a,b);
    printf("\nTotal : %d",a);
    return 0;
}

int add(int x,int y)
{
    return x+y;
}
```

Output

```
Enter The Value of A & B : 2
9

Total : 11
```

Q 14 what is string? Explain string handling function with example.

A **string** is any series of characters that are interpreted literally by a script. For example, "hello world" and "LKJH019283" are both examples of strings.

C programming language provides a set of pre-defined functions called **string handling functions** to work with string values.

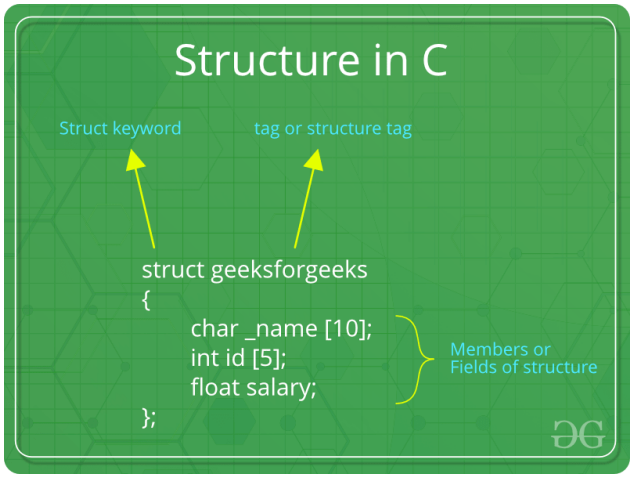
The string handling functions are defined in a header file called **string.h**.

Whenever we want to use any string handling function we must include the header file called **string.h**

Function	Syntax (or) Example	Description
strcpy()	strcpy(string1, string2)	Copies string2 value into string1
strncpy()	strncpy(string1, string2, 5)	Copies first 5 characters string2 into string1
strlen()	strlen(string1)	returns total number of characters in string1
strcat()	strcat(string1,string2)	Appends string2 to string1
strncat()	strncpy(string1, string2, 4)	Appends first 4 characters of string2 to string1
strcmp()	strcmp(string1, string2)	Returns 0 if string1 and string2 are the same; less than 0 if string1<string2; greater than 0 if string1>string2
strncmp()	strncmp(string1, string2, 4)	Compares first 4 characters of both string1 and string2
strcmpi()	strcmpi(string1,string2)	Compares two strings, string1 and string2 by ignoring case (upper or lower)
stricmp()	stricmp(string1, string2)	Compares two strings, string1 and string2 by ignoring case (similar to strcmpi())
strlwr()	strlwr(string1)	Converts all the characters of string1 to lower case.
strupr()	strupr(string1)	Converts all the characters of string1 to upper case.
strdup()	string1 = strdup(string2)	Duplicated value of string2 is assigned to string1
strchr()	strchr(string1, 'b')	Returns a pointer to the first occurrence of character 'b' in string1
strrchr()	'strrchr(string1, 'b')	Returns a pointer to the last occurrence of character 'b' in string1
strstr()	strstr(string1, string2)	Returns a pointer to the first occurrence of string2 in string1
strset()	strset(string1, 'B')	Sets all the characters of string1 to given character 'B'.
strnset()	strnset(string1, 'B', 5)	Sets first 5 characters of string1 to given character 'B'.
strrev()	strrev(string1)	It reverses the value of string1

Q 15 Explain structure in Detail.

A structure is a key word that create user defined data type in C/C++. A structure creates a data type that can be used to group items of possibly different types into a single type.



How to create a structure?

'struct' keyword is used to create a structure. Following is an example.

```
struct address
{
   char name[50];
   char street[100];
   char city[50];
   char state[20];
   int pin;
};
```

How to declare structure variables?

A structure variable can either be declared with structure declaration or as a separate declaration like basic types.

```c
// A variable declaration with structure declaration.
struct Point
{
   int x, y;
} p1;  // The variable p1 is declared with 'Point'


// A variable declaration like basic data types
struct Point
{
   int x, y;
};

int main()
{
   struct Point p1;  // The variable p1 is declared like a normal variabl
}
```

How to initialize structure members?

Structure members cannot be initialized with declaration. For example the following C program fails in compilation.

```c
struct Point
{
   int x = 0;  // COMPILER ERROR:  cannot initialize members here
   int y = 0;  // COMPILER ERROR:  cannot initialize members here
};
```

The reason for above error is simple, when a datatype is declared, no memory is allocated for it. Memory is allocated only when variables are created.

Structure members can be initialized using curly braces '{}'. For example, following is a valid initialization.

```c
struct Point
{
   int x, y;
};

int main()
{
   // A valid initialization. member x gets value 0 and y
   // gets value 1.  The order of declaration is followed.
   struct Point p1 = {0, 1};
}
```

How to access structure elements?

Structure members are accessed using dot (.) operator.

```c
#include<stdio.h>

struct Point
{
   int x, y;
};

int main()
{
   struct Point p1 = {0, 1};

   // Accessing members of point p1
   p1.x = 20;
   printf ("x = %d, y = %d", p1.x, p1.y);

   return 0;
}
```

Output:

```
x = 20, y = 1
```

What is an array of structures?

Like other primitive data types, we can create an array of structures.

```c
#include<stdio.h>

struct Point
{
   int x, y;
};

int main()
{
   // Create an array of structures
   struct Point arr[10];

   // Access array members
   arr[0].x = 10;
   arr[0].y = 20;

   printf("%d %d", arr[0].x, arr[0].y);
   return 0;
}
```

Output:

```
10 20
```

What is a structure pointer?

Like primitive types, we can have pointer to a structure. If we have a pointer to structure, members are accessed using arrow (->) operator.

```c
#include<stdio.h>

struct Point
{
   int x, y;
};

int main()
{
   struct Point p1 = {1, 2};

   // p2 is a pointer to structure p1
   struct Point *p2 = &p1;

   // Accessing structure members using structure pointer
   printf("%d %d", p2->x, p2->y);
   return 0;
}
```

Output:

```
1 2
```

Q 16 Difference between Structure and Union

	STRUCTURE	UNION
Keyword	The keyword **struct** is used to define a structure	The keyword **union** is used to define a union.
Size	When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is **greater than or equal to the sum of sizes of its members.**	when a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of **union is equal to the size of largest member.**
Memory	Each member within a structure is assigned unique storage area of location.	Memory allocated is shared by individual members of union.
Value Altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the member will alter other member values.
Accessing members	Individual member can be accessed at a time.	Only one member can be accessed at a time.
Initialization of Members	Several members of a structure can initialize at once.	Only the first member of a union can be initialized.

Q 17 Explain Dynamic memory allocation. Functions of Dynamic Memory Allocation.

Dynamic memory allocation means memory can be allocated or de-allocated as per the requirement at run time.

Memory can be allocated as well as de-allocated as per necessity. No need to initially occupy large memory. It avoids the memory shortage as well as memory wastage. Dynamic memory management is considered as manual memory management. This concept helps to obtain more memory when required as well as release it when not necessary.

Functions Dynamic Memory Allocation

1) malloc()

2) calloc()

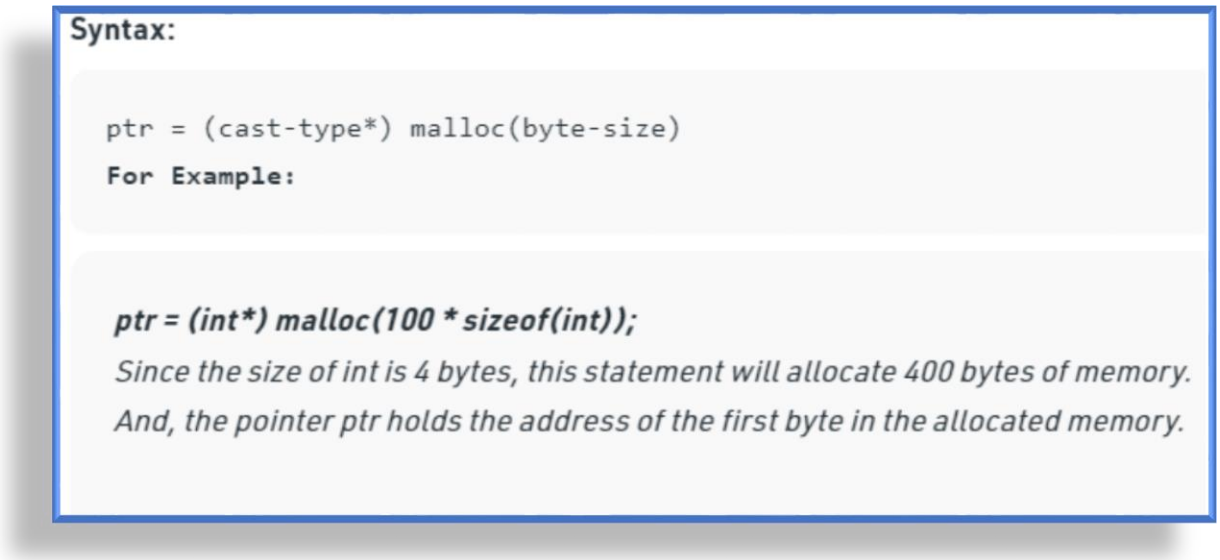
3) free()

4) realloc()

1) malloc()

The **"malloc"** or **"memory allocation"** method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form.

ptr = (cast-type*)malloc(bite-size);



2) calloc()

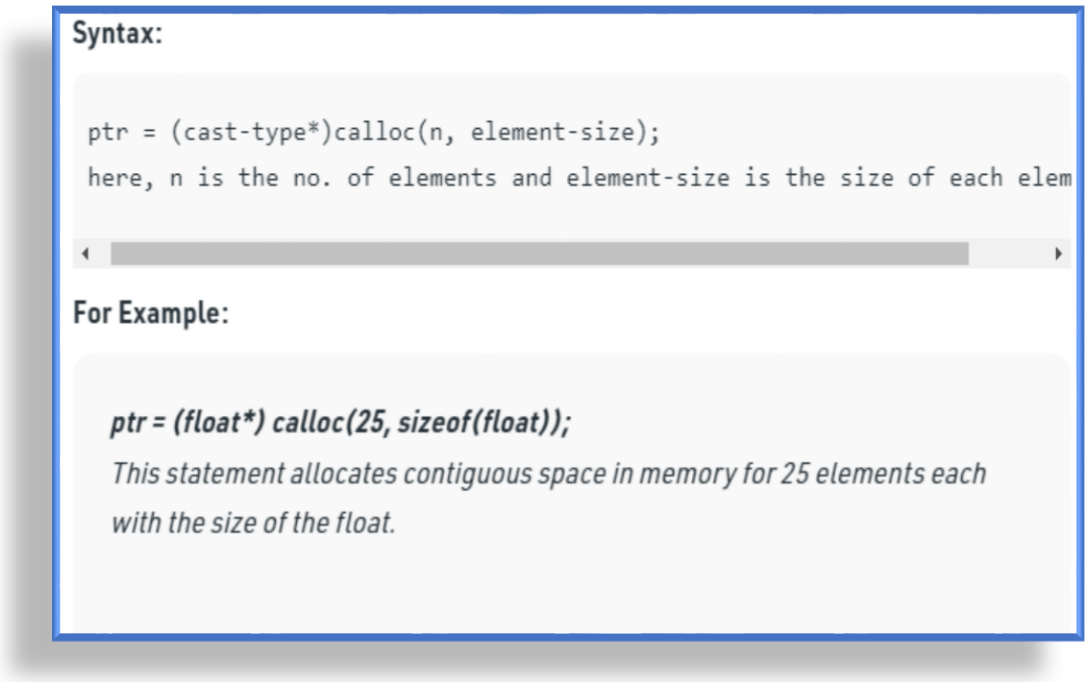
"calloc" or **"contiguous allocation"** method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. it is very much similar to malloc() but has two different points and these are:

It initializes each block with a default value '0'.

It has two parameters or arguments as

compare to malloc()

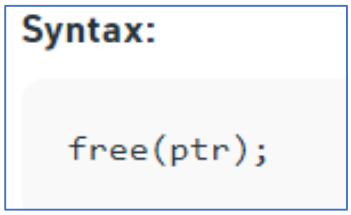
ptr = (cast-type*)calloc(n, element-size);



3) free()

"free" method in C is used to dynamically **de-allocate** the memory. The memory allocated using functions malloc() and calloc() is not de-allocated on their own. Hence the free() method is used, whenever the dynamic memory allocation takes place. It helps to reduce wastage of memory by freeing it.

Free(ptr);



4) realloc()

"realloc" or **"re-allocation"** method in C is used to dynamically change the memory allocation of a previously allocated memory. If the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**. Re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

ptr = relloc(ptr, newsize);

Syntax:

```
ptr = realloc(ptr, newSize);

where ptr is reallocated with new size 'newSize'.
```

A file management system is used for file maintenance (or management) operations.

It is is a type of software that manages data files in a computer system.

File handling is to create, update, read, and delete the files stored on the local file system through our program. The following operations can be performed on a file.

- Creation of the new file
- Opening an existing file
- Reading from the file
- Writing to the file
- Deleting the file

Functions for file handling

There are many functions in the C library to open, read, write, search and close the file. A list of file functions are given below:

No.	Function	Description
1	fopen()	opens new or existing file
2	fprintf()	write data into the file
3	fscanf()	reads data from the file
4	fputc()	writes a character into the file
5	fgetc()	reads a character from file
6	fclose()	closes the file
7	fseek()	sets the file pointer to given position
8	fputw()	writes an integer to file
9	fgetw()	reads an integer from file
10	ftell()	returns current position
11	rewind()	sets the file pointer to the beginning of the file

Q 18 Explain input output function with example.

When we say **Input**, it means to feed some data into a program. An input can be given in the form of a file or from the command line. C programming provides a set of built-in functions to read the given input and feed it to the program as per requirement.

When we say **Output**, it means to display some data on screen, printer, or in any file. C programming provides a set of built-in functions to output the data on the computer screen as well as to save it in text or binary files.

getchar() and putchar() Functions

The **int getchar(void)** function reads the next available character from the screen and returns it as an integer. This function reads only single character at a time. You can use this method in the loop in case you want to read more than one character from the screen.

The **int putchar(int c)** function puts the passed character on the screen and returns the same character. This function puts only single character at a time. You can use this method in the loop in case you want to display more than one character on the screen.

following example –

```
#include <stdio.h>
int main( ) {

   int c;

   printf( "Enter a value :");
   c = getchar( );

   printf( "\nYou entered: ");
   putchar( c );

   return 0;
}
```

When the code is compiled and executed, it waits for you to input some text. When you enter a text and press enter, then the program proceeds and reads only a single character and displays.

```
$./a.out
Enter a value : this is test
You entered: t
```

The gets() and puts() Functions

The **char *gets(char *s)** function reads a line from **stdin** into the buffer pointed to by **s** until either a terminating newline or EOF (End of File).

The **int puts(const char *s)** function writes the string 's' and 'a' trailing newline to **stdout**.

```c
#include <stdio.h>
int main( ) {

   char str[100];

   printf( "Enter a value :");
   gets( str );

   printf( "\nYou entered: ");
   puts( str );

   return 0;
}
```

When the code is compiled and executed, it waits for you to input some text. When you enter a text and press enter, then the program proceeds and reads the complete line till end, and displays it as follows −

```
$./a.out
Enter a value : this is test
You entered: this is test
```

The scanf() and printf() Functions

The **int scanf(const char *format, ...)** function reads the input from the standard input stream **stdin** and scans that input according to the **format** provided.

The **int printf(const char *format, ...)** function writes the output to the standard output stream **stdout** and produces the output according to the format provided.

The **format** can be a simple constant string, but you can specify %s, %d, %c, %f, etc., to print or read strings, integer, character or float respectively.

```c
#include <stdio.h>
int main( ) {

   char str[100];
   int i;

   printf( "Enter a value :");
   scanf("%s %d", str, &i);

   printf( "\nYou entered: %s %d ", str, i);

   return 0;
}
```

When the code is compiled and executed, it waits for you to input some text. When you enter a text and press enter, then program proceeds and reads the input and displays it as follows –

```
$./a.out
Enter a value : seven 7
You entered: seven 7
```

Q 19 Print following Pattern using loop.


```c
#include <stdio.h>
int main() {
   int i, j, rows;
   printf("Enter the number of rows: ");
   scanf("%d", &rows);
   for (i = 1; i <= rows; ++i) {
      for (j = 1; j <= i; ++j) {
         printf("* ");
      }
      printf("\n");
   }
   return 0;
}
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```c
#include <stdio.h>
int main() {
   int i, j, rows;
   printf("Enter the number of rows: ");
   scanf("%d", &rows);
   for (i = 1; i <= rows; ++i) {
      for (j = 1; j <= i; ++j) {
         printf("%d ", j);
      }
      printf("\n");
   }
   return 0;
}
```

```
A
B B
C C C
D D D D
E E E E E
```

```c
#include <stdio.h>
int main() {
   int i, j;
   char input, alphabet = 'A';
   printf("Enter an uppercase character you want to print in the last row: ");
   scanf("%c", &input);
   for (i = 1; i <= (input - 'A' + 1); ++i) {
      for (j = 1; j <= i; ++j) {
         printf("%c ", alphabet);
      }
      ++alphabet;
      printf("\n");
   }
   return 0;
}
```

```
* * * * *
* * * *
* * *
* *
*
```

```c
#include <stdio.h>
int main() {
   int i, j, rows;
   printf("Enter the number of rows: ");
   scanf("%d", &rows);
   for (i = rows; i >= 1; --i) {
      for (j = 1; j <= i; ++j) {
         printf("* ");
      }
      printf("\n");
   }
   return 0;
}
```

Write a program to evaluate the following series

1. 1^2+2^2+3^2+……+n^2

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,i,sum=0;
    printf("\n Enter Value of n : ");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
     sum=sum+(i*i);
    }
    printf("\n Sum of Series = %d",sum);
      getch();
}
```

Output

```
Enter Value of n : 5

Sum of Series = 55
```

2. 1+1/2+1/3+1/4+....+1/n.

```
#include<stdio.h>
#include<conio.h>
void main()
{
   int n,i;
   float sum=0;
   printf("\n Enter Value of n : ");
```

```
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        sum=sum+(1.0/i);
    }
    printf("\n Sum of Series = %f",sum);
       getch();
}
```

Output

```
Enter Value of n : 5

Sum of Series = 2.283334
```

Q21 Explain Pointer in C programming .

Pointers in C are easy and fun to learn. Some C programming tasks are performed more easily with pointers, and other tasks, such as dynamic memory allocation, cannot be performed without using pointers. So it becomes necessary to learn pointers to become a perfect C programmer. Let's start learning them in simple and easy steps.

As you know, every variable is a memory location and every memory location has its address defined which can be accessed using ampersand (&) operator, which denotes an address in memory. Consider the following example, which prints the address of the variables defined −

What are Pointers?

A pointer is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address. The general form of a pointer variable declaration is

```
type *var-name;
```

Here, type is the pointer's base type; it must be a valid C data type and var-name is the name of the pointer variable. The asterisk * used to declare a pointer is the same asterisk used for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer. Take a look at some of the valid pointer declarations –

```
int *ip; /* pointer to an integer */
double *dp; /* pointer to a double */
float *fp; /* pointer to a float */
char *ch /* pointer to a character */
```

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to.

How to Use Pointers?

There are a few important operations, which we will do with the help of pointers very frequently. (a) We define a pointer variable, (b) assign the address of a variable to a pointer and (c) finally access the value at the address available in the pointer variable. This is done by using unary operator * that returns the value of the variable located at the address specified by its operand. The following example makes use of these operations –

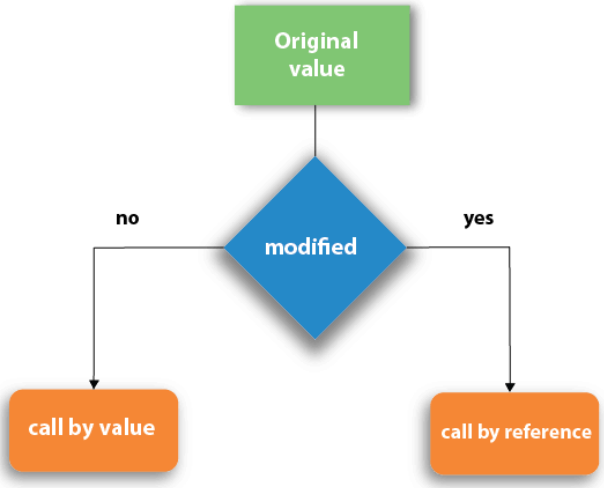
```
#include <stdio.h>

int main () {

   int  var = 20;   /* actual variable declaration */
   int  *ip;        /* pointer variable declaration */

   ip = &var;  /* store address of var in pointer variable*/

   printf("Address of var variable: %x\n", &var  );

   /* address stored in pointer variable */
   printf("Address stored in ip variable: %x\n", ip );

   /* access the value using the pointer */
   printf("Value of *ip variable: %d\n", *ip );

   return 0;
}
```

When the above code is compiled and executed, it produces the following result –

```
Address of var variable: bffd8b3c
Address stored in ip variable: bffd8b3c
Value of *ip variable: 20
```

Q22 Explain Call by value and Call by reference in C

There are two methods to pass the data into the function in C language, i.e., call by value and call by reference.



Call by value in C

- In call by value method, the value of the actual parameters is copied into the formal parameters. In other words, we can say that the value of the variable is used in the function call in the call by value method.
- In call by value method, we can not modify the value of the actual parameter by the formal parameter.
- In call by value, different memory is allocated for actual and formal parameters since the value of the actual parameter is copied into the formal parameter.
- The actual parameter is the argument which is used in the function call whereas formal parameter is the argument which is used in the function definition.

```c
#include<stdio.h>
void change(int num) {
   printf("Before adding value inside function num=%d \n",num);
   num=num+100;
   printf("After adding value inside function num=%d \n", num);
}
int main() {
   int x=100;
   printf("Before function call x=%d \n", x);
   change(x);//passing value in function
   printf("After function call x=%d \n", x);
return 0;
}
```

Output

```
Before function call x=100
Before adding value inside function num=100
After adding value inside function num=200
After function call x=100
```

Call by reference in C

- In call by reference, the address of the variable is passed into the function call as the actual parameter.
- The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed.
- In call by reference, the memory allocation is similar for both formal parameters and actual parameters. All the operations in the function are performed on the value stored at the address of the actual parameters, and the modified value gets stored at the same address.

```c
#include<stdio.h>
void change(int *num) {
   printf("Before adding value inside function num=%d \n",*num);
   (*num) += 100;
   printf("After adding value inside function num=%d \n", *num);
}
int main() {
   int x=100;
   printf("Before function call x=%d \n", x);
   change(&x);//passing reference in function
   printf("After function call x=%d \n", x);
return 0;
}
```

Output

```
Before function call x=100
Before adding value inside function num=100
After adding value inside function num=200
After function call x=200
```

Difference between call by value and call by reference in c

No.	Call by value	Call by reference
1	A copy of the value is passed into the function	An address of value is passed into the function
2	Changes made inside the function is limited to the function only. The values of the actual parameters do not change by changing the formal parameters.	Changes made inside the function validate outside of the function also. The values of the actual parameters do change by changing the formal parameters.
3	Actual and formal arguments are created at the different memory location	Actual and formal arguments are created at the same memory location